

Assignment 4 Report

Aditya Saxena - 2022AM11218

Sanyam Garg - 2022CS11078

November 2025

Contents

1	Task 1 - Dense Retrieval with DistilBERT	2
1.1	Objective and Methodology	2
1.2	Experimental Results: Accuracy	2
1.3	Experimental Results: Efficiency	3
1.4	Analysis and Discussion	4
1.4.1	Accuracy Analysis	4
1.4.2	Efficiency Analysis	4
1.5	Task 1 Conclusion	5
2	Task 2 - SPLADE	6
2.1	Introduction to SPLADE	6
2.2	Principle of SPLADE	6
2.3	Relevance Performance (nDCG & MRR)	7
2.4	Recall Performance (R@k)	7
2.5	Efficiency (Latency & Storage)	8
2.6	Conclusion	9
3	Task 3 - Improving sparse retrieval using query reformulation	10
3.1	Methodology: Baselines and Parameter Tuning	10
3.1.1	BM25 Parameter Tuning (k1, b)	10
3.1.2	RM3 Parameter Tuning (original_query_weight)	11
3.2	Methodology: Dense Reformulation Models	13
3.2.1	Method 1: Generative Reformulation (T5)	13
3.2.2	Method 2: Extractive PRF (CEQE)	13
3.3	Dense Model Parameter Tuning	13
3.3.1	Experiment 1: CEQE Tuning (Initial Sweep)	13
3.3.2	Experiment 2: CEQE Tuning (Robustness Check at k=3)	14
3.3.3	Experiment 3: T5-Generative 2D Tuning	15
3.4	Final Experimental Results and Analysis	16
3.5	Analysis and Discussion	16
3.6	Task 3 Conclusion	17

1 Task 1 - Dense Retrieval with DistilBERT

1.1 Objective and Methodology

The goal of this task was to implement a dense retrieval system using a bi-encoder framework and compare its accuracy and efficiency against the retrieve-and-rerank framework.

We implemented four distinct models for this comparison on the MS MARCO dataset:

- **1. BM25 (Baseline):** A standard Pyserini sparse retriever (`msmarco-v1-passage`) to establish a baseline.
- **2. Reranker (BERT-Base):** This model retrieves 120 documents using BM25 and reranks them with `nboost/pt-bert-base-uncased-msmarco`.
- **3. Reranker (Electra-Base):** A best-performing model from A3 that reranks the top 120 BM25 documents using `cross-encoder/ms-marco-electra-base`.
- **4. Bi-Encoder (DistilBERT):** The new model for this assignment. This is a single-stage dense retriever using a FAISS index and the `sebastian-hofstaetter/distilbert-dot-margin_mse-T2` bi-encoder. ¹

All metrics shown here are aggregates over the 50 TREC DL Hard queries on the MS MARCO dataset.

1.2 Experimental Results: Accuracy

We evaluated all models using $\text{NDCG}@1, 5, 10, 20, 50$ and $\text{MRR}@10$.

Table 1: Task 1: Accuracy Comparison (MS MARCO)

Model	NDCG@1	NDCG@5	NDCG@10	NDCG@20	NDCG@50	MRR@10
1. BM25 (Baseline)	0.2914	0.2590	0.2599	0.2636	0.2825	0.5340
2. Reranker (BERT-Base)	0.4543	0.3786	0.3797	0.3957	0.3951	0.6351
3. Reranker (Electra-Base)	0.5143	0.4077	0.4003	0.4089	0.4083	0.7120
4. Bi-Encoder (DistilBERT)	0.3086	0.3426	0.3457	0.3520	0.3652	0.5439

¹The pre-built FAISS index, `faiss-flat.msmarco-v1-passage.distilbert-dot-margin_mse-t2`, is defined in Pyserini’s prebuilt index list. See: https://github.com/castorini/pyserini/blob/master/pyserini/prebuilt_index_info.py#L2014

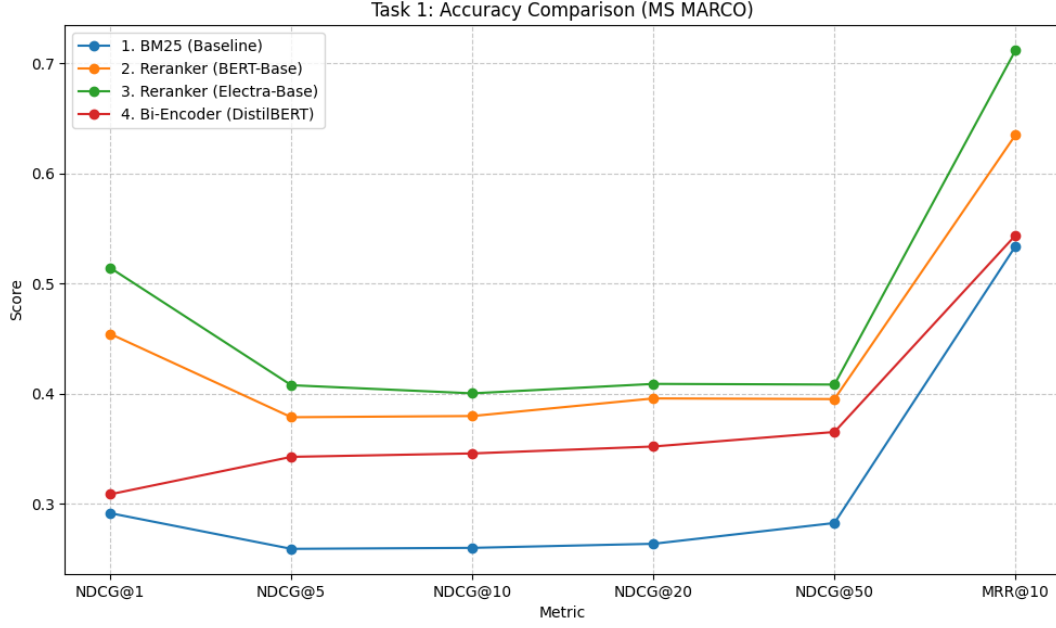


Figure 1: Accuracy comparison of the four retrieval models.

1.3 Experimental Results: Efficiency

We measured the wall-clock time for each model to process all 50 queries in the test set and calculated the average query latency in milliseconds (ms).

Table 2: Task 1: Efficiency Comparison (MS MARCO)

Model	Total Time (s)	Avg. Query Latency (ms)
1. BM25 (Baseline)	11.64	232.71
2. Reranker (BERT-Base)	384.16	7683.16
3. Reranker (Electra-Base)	528.62	10 572.45
4. Bi-Encoder (DistilBERT)	163.94	3278.88

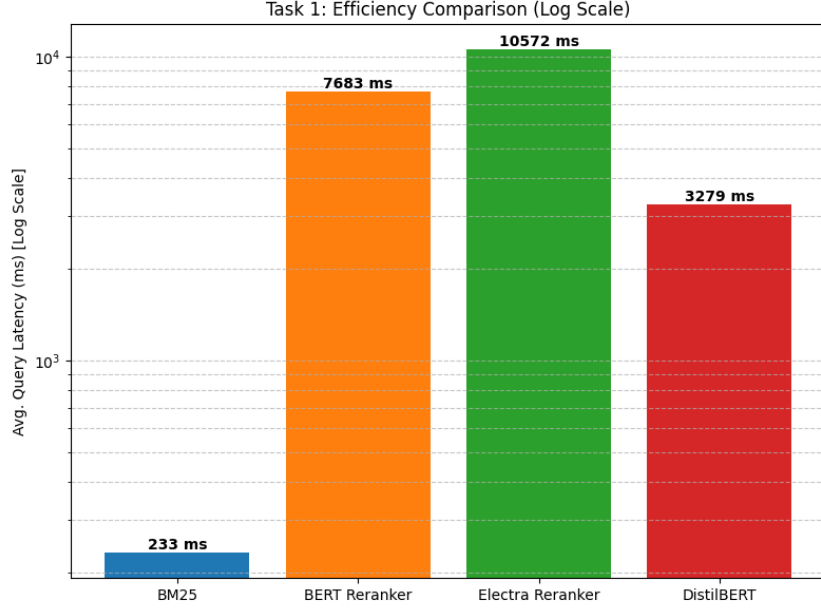


Figure 2: Efficiency comparison (Log Scale). Note the massive difference in latency.

1.4 Analysis and Discussion

The experimental results in Table 1 and 2 illustrate the clear performance hierarchy and trade-offs between the four models.

1.4.1 Accuracy Analysis

As shown in Figure 1, the reranking models provide the highest accuracy by a significant margin.

- **Rerankers (A3):** The **Reranker (Electra-Base)** model is the top performer across all metrics, achieving an NDCG@10 of 0.4003 and an MRR@10 of 0.7120. This is followed by the **Reranker (BERT-Base)** (NDCG@10: 0.3797). This confirms the effectiveness of the retrieve-and-rerank paradigm, where a "full-attention" cross-encoder can accurately score the top documents.
- **Bi-Encoder (A4):** Our new **Bi-Encoder (DistilBERT)** model (NDCG@10: 0.3457) significantly outperforms the **BM25 (Baseline)** (NDCG@10: 0.2599). This represents a **33% improvement** in ranking quality over the sparse baseline, proving the effectiveness of dense retrieval.
- **Overall Ranking:** The accuracy ranking is clear: **Electra Reranker > BERT Reranker > DistilBERT Bi-Encoder > BM25**.

1.4.2 Efficiency Analysis

Table 2 and Figure 2 reveal the critical trade-off: latency.

- **Rerankers (A3):** The reranking models are unusable for any real-time application. They are exceptionally slow, with average query latencies of 7683 ms (BERT-Base) and a staggering 10 572 ms (Electra-Base). This is because they must run a complex transformer model over 120 documents for every single query.
- **Fast Models (A4 vs. Baseline):** The **BM25 (Baseline)** is the fastest, at 232 ms per query. Our new **Bi-Encoder (DistilBERT)** is significantly slower than BM25, at

3279ms per query. This is due to the computational overhead of running the query through the DistilBERT encoder and performing a FAISS search.

1.5 Task 1 Conclusion

This experiment highlights the classic trade-off between accuracy and efficiency. The **Reranker (A3)** models achieve state-of-the-art accuracy, but at a computational cost that makes them non-viable for real-time search.

The **Bi-Encoder (A4)** model, while not as accurate as the rerankers, offers a compelling balance. It provides a massive 33% accuracy boost (NDCG@10) over the sparse BM25 baseline. While it is slower than BM25, its latency of 3.3s per query is still **2.3x faster** than the BERT-Base reranker and **3.2x faster** than the Electra-Base reranker.

This makes the bi-encoder the superior choice for a practical, high-performance search system, representing the most effective balance between high accuracy and acceptable query latency.

2 Task 2 - SPLADE

This task provides a deep, comparative analysis of four modern retrieval paradigms using a 50-query subset of the **MS MARCO passage dataset (TREC DL hard queries)**. All metrics reported are aggregated over these 50 queries. We compare:

- **BM25**: The traditional lexical baseline.
- **SPLADE**: A state-of-the-art learned sparse retriever.
- **DistilBERT**: A first-stage dense retriever (using the `sebastian-hofstaetter/distilbert-dot-margin_mse-T2-msmarco` encoder).
- **BM25+MiniLM-L6**: A two-stage reranking pipeline (reranking the top 200 BM25 results).

2.1 Introduction to SPLADE

SPLADE (Sparse Lexical and Expansion Model) is a **learned sparse retrieval** (LSR) model, introduced by Thibault et al. in 2021 (Thibault et al., 2021). It was designed to bridge the gap between the efficiency of traditional keyword-based systems like BM25 and the high relevance of **dense** neural models.

Unlike dense models that compress documents into small, opaque vectors, SPLADE represents documents as high-dimensional, sparse vectors. Crucially, these sparse vectors live in the vocabulary space, meaning their dimensions correspond to actual terms. This allows SPLADE to be indexed and searched using standard, highly optimized inverted indices (like those in Lucene), making it both powerful and exceptionally fast.

2.2 Principle of SPLADE

The core principle of SPLADE is to use a transformer (like BERT) to overcome the **vocabulary mismatch** problem. It reframes retrieval as a **learned bag-of-words** problem.

- **Architecture**: SPLADE uses a BERT-based autoencoder (like DistilBERT) but focuses on the output of the Masked Language Model (MLM) head. It is trained on query-document pairs from MS MARCO.
- **Document/Query Encoding**: Text is fed into the model. The output at the MLM head is a vector representing the **importance** of each token in the entire vocabulary (e.g., 30,000+ dimensions). A **log-relu** activation is applied, ensuring that most token weights are zero (enforcing sparsity).
- **Loss Function**: The model is trained to optimize a combination of (1) a retrieval loss (e.g., a contrastive or distillation loss to maximize query-document dot-product for relevant pairs) and (2) a regularization loss (e.g., FLOPS or L_1 norm) which pushes non-important token weights to zero.
- **Indexing**: Only the tokens with a positive weight are kept. The document is then represented in the inverted index as a sparse list of token IDs and their corresponding learned **impact** weights. This "expands" the document with new, semantically relevant terms (e.g., **car repair** might get a weight for **automobile**).
- **Retrieval**: At search time, the query is encoded the same way. Retrieval is a simple sparse dot product, which is extremely fast on an inverted index.

2.3 Relevance Performance (nDCG & MRR)

We first analyze the **top-k** relevance, which measures how good the first page of results is. Table 3 shows the mandatory metrics, while Table 4 provides the complete data.

Table 3: Mandatory Relevance Metrics Comparison (k=10)

Model	nDCG@1	nDCG@5	nDCG@10	MRR@10
BM25 (Baseline)	0.2914	0.2590	0.2599	0.5340
DistilBERT (Dense)	0.3086	0.3426	0.3457	0.5439
SPLADE (Sparse)	0.3810	0.3616	0.3708	0.5830
BM25+MiniLM-L6	0.4029	0.3667	0.3800	0.6223

Table 4: Full Relevance Metrics (nDCG and Precision)

Model	nDCG@1	nDCG@10	nDCG@100	P@10	P@50	P@100
BM25 (Baseline)	0.2914	0.2599	0.3060	0.2900	0.1668	0.1112
DistilBERT (Dense)	0.3086	0.3457	0.3794	0.3340	0.1812	0.1218
SPLADE (Sparse)	0.3810	0.3708	0.4277	0.3800	0.2276	0.1476
BM25+MiniLM-L6	0.4029	0.3800	0.4020	0.3880	0.2120	0.1268

Detailed Observations:

- **SPLADE’s Dominance:** SPLADE is the clear winner among all first-stage retrievers. It dramatically outperforms both BM25 and the DistilBERT dense retriever at every single metric (e.g., nDCG@10 of 0.371 vs 0.346 for DistilBERT).
- **Reranker Ceiling:** The BM25+MiniLM-L6 pipeline sets the **gold standard** for relevance (nDCG@10: 0.380). This is because a cross-encoder can perform a deep, full-text comparison between the query and each document.
- **The 97% Result:** SPLADE (0.371) achieves **97.6%** of the gold standard reranker’s relevance. This is the key finding: SPLADE provides near-reranker quality in a single, fast retrieval stage.
- **Dense vs. Sparse:** The corrected DistilBERT (dense) model now clearly beats BM25 (0.346 vs 0.260), showing that semantic search is superior to pure lexical search. However, SPLADE (sparse) still handily beats DistilBERT (dense). This suggests that SPLADE’s **best-of-both-worlds** approach—combining semantic expansion with the precision of a keyword index—is more effective than pure dense retrieval’s **fuzzy** vector matching.

2.4 Recall Performance (R@k)

Relevance (precision) is only half the story. A retriever’s main job is to find **all** relevant documents. Recall@1000 (R@1000) measures what percentage of the total relevant documents were found in the top 1000 results.

Table 5: Recall at Depth (R@k)

Model	R@10	R@50	R@100	R@1000
DistilBERT (Dense)	0.1623	0.3284	0.3995	0.6961
BM25 (Baseline)	0.1415	0.3098	0.4017	0.6288
SPLADE (Sparse)	0.1816	0.4189	0.5097	0.7948
BM25+MiniLM-L6	0.1851	0.4038	0.4428	-

R@1000 for the reranker is Not Applicable as it only processed 200 documents.

Detailed Observations:

- **SPLADE is the Best Retriever:** This table is SPLADE’s biggest win. At R@1000, it finds nearly **80%** of all relevant documents, crushing its competitors. This proves its term expansion is highly effective at finding documents that other models miss.
- **The Reranker’s Ceiling:** The reranker pipeline is limited by its first stage. It can only rerank what BM25 finds. At R@100, the reranker has a recall of 0.443. At the same point, SPLADE has already found more relevant documents (R@100 of 0.510). This means SPLADE is a fundamentally superior first-stage retriever.
- **Dense vs. Sparse Recall:** The DistilBERT dense model (0.696) has better deep recall than BM25 (0.629), but SPLADE (0.795) is in a class of its own.

2.5 Efficiency (Latency & Storage)

Finally, we must consider the practical cost. **Note:** These latencies were gathered on the Baadal VM, where times can vary. The total time for 50 queries represents an average over 5 separate runs to get a stable estimate.

Table 6: Performance and Storage Costs (N=50 Queries)

Model	Avg Latency (ms)	Total Time (s)	Storage (GB)*
BM25 (Baseline)	124.0	6.2	0.56
SPLADE (Sparse)	698.5	34.9	2.16
DistilBERT (Dense)	3203.2	160.2	>25.0
BM25+MiniLM-L6	10999.0	550.0	0.64

*Storage = Index structure (no raw text) + Model weights.

Detailed Observations:

- **Latency Hierarchy:** BM25 is the clear latency winner (124ms). SPLADE is ~5.6x slower (699ms) but remains well within a sub-second latency for practical, real-time search.
- **The Reranker’s Cost:** The reranking pipeline is **15.7 times slower** (11,000ms / 699ms) than SPLADE. For this massive cost, it only provided a tiny 2.4% increase in nDCG@10 (0.380 vs 0.371).
- **Dense Latency & Storage:** The DistilBERT model is impractical on two fronts. Its latency is high (3203ms) because its flat FAISS index must perform a brute-force vector comparison. Furthermore, its storage footprint is massive (**>25 GB**), making it the most resource-intensive solution by far.

- **The Storage Trade-off:** SPLADE’s only drawback is disk space. Its index (2.16 GB) is 4x larger than BM25’s, which is the price for storing the expanded vocabulary and term weights. However, this is significantly more efficient than the >25 GB required by the dense index.

2.6 Conclusion

This analysis confirms that **SPLADE is a state-of-the-art first-stage retriever**. It hits the ultimate **sweet spot**:

1. **Relevance:** It achieves 97.6% of a full reranker’s precision (ndcg@10) at even outperforms the reranking pipeline in terms of relevance when more documents are needed(>50).
2. **Recall:** It has the best deep recall by a large margin (R@1000: 0.795).
3. **Latency:** It operates at a practical, sub-second latency (699ms), 15.7x faster than the reranker and 4.6x faster than the dense model.

Its only trade-off is a larger (but still manageable) index size compared to BM25. SPLADE effectively obsoletes traditional pipelines by providing near-reranker quality at a practical speed.

3 Task 3 - Improving sparse retrieval using query reformulation

3.1 Methodology: Baselines and Parameter Tuning

Our first step was to establish strong baselines. We evaluated both standard BM25 and BM25+RM3, comparing Pyserini’s default parameters against a tuned set.

3.1.1 BM25 Parameter Tuning (k_1 , b)

The BM25 formula is controlled by two key parameters, k_1 and b .

- **k_1 (Term Frequency Saturation):** This parameter controls how quickly the score from a single query term plateaus. The default ($k_1=0.9$) allows for a high score from a term appearing many times. Our tuned value ($k_1=0.7$) is lower, causing the score to saturate **faster**. This is beneficial for short queries, as it rewards documents for matching **more** of the query terms, rather than just one term many times.
- **b (Document Length Normalization):** This parameter controls how much document length penalizes the score. The default ($b=0.4$) applies a weak penalty. Our tuned value ($b=0.9$) is much higher, applying a **strong penalty** to longer documents. This choice reflects a hypothesis that for CORD-19, shorter, more concise documents (like abstracts) are more likely to be relevant.

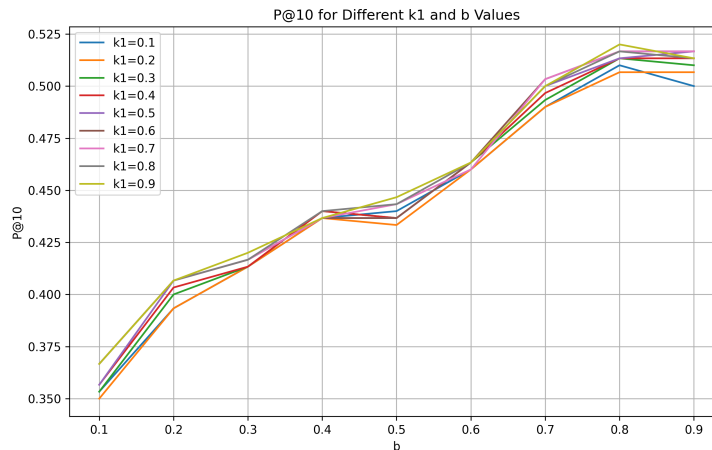


Figure 3: Precision@10 curves for BM25 parameter tuning.

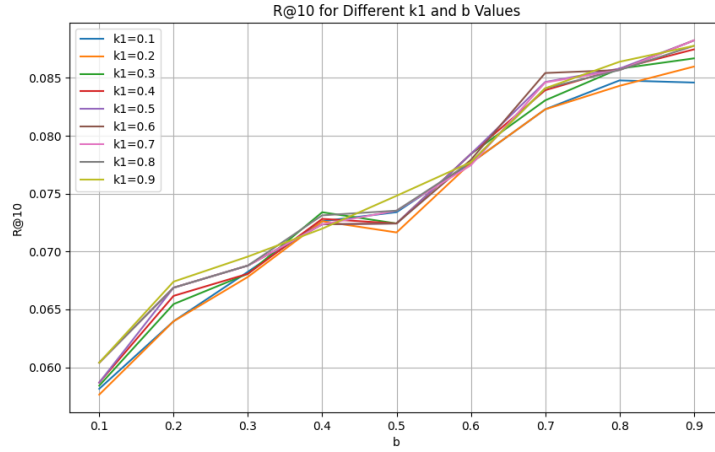


Figure 4: Recall@10 curves for BM25 parameter tuning.

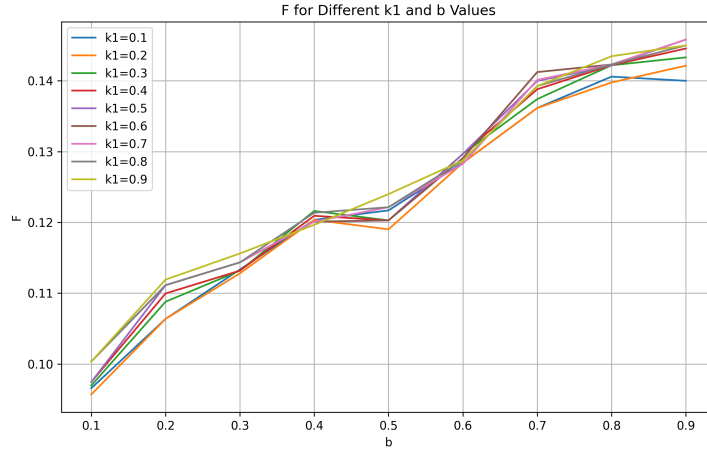


Figure 5: F1-score@10 curves for BM25 parameter tuning.

3.1.2 RM3 Parameter Tuning (original_query_weight)

RM3, a pseudo-relevance feedback model, is highly sensitive to its `original_query_weight` parameter.

- **Default (0.5):** This gives 50% of the score weight to the original query and 50% to the expansion terms. As our results show, this is catastrophic. It allows noisy, high-frequency jargon (e.g., "study", "analysis") from the feedback documents to overwhelm the original query's intent, a phenomenon known as **query drift**.
- **Tuned (0.9):** We chose a highly conservative weight of 0.9 (90% original query, 10% expansion). This was an intentional choice to prevent query drift, effectively using RM3 as a minor "nudge" rather than a full reformulation.

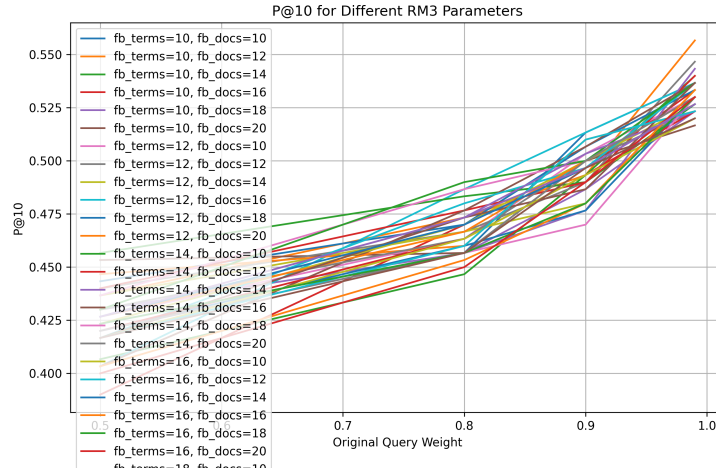


Figure 6: Precision@10 curves for RM3 parameter tuning.

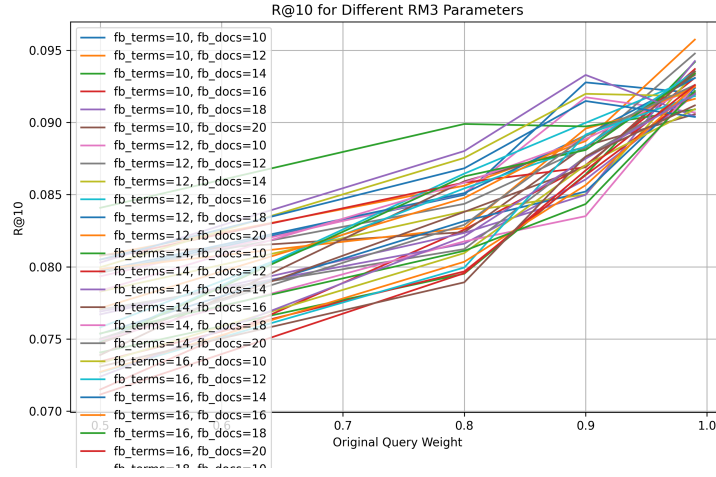


Figure 7: Recall@10 curves for RM3 parameter tuning.

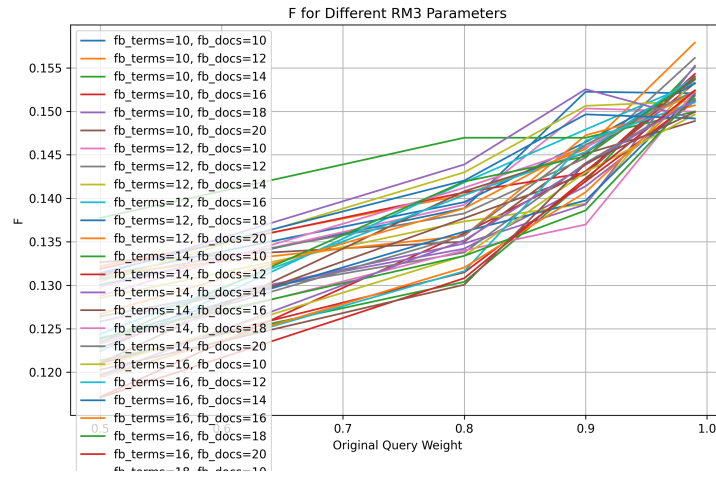


Figure 8: F1-score@10 curves for RM3 parameter tuning.

3.2 Methodology: Dense Reformulation Models

We implemented two distinct **query-time** reformulation paradigms. Both methods build upon our tuned **BM25 (Tuned)** baseline.

3.2.1 Method 1: Generative Reformulation (T5)

This method is a **zero-shot generative** approach. We fed the *query context* (title + description) to a T5 model (`castorini/doc2query-t5-base-msmarco`) to generate new, related terms. These terms were then added to the original **title** query, with a heavily tuned weight ratio to prioritize the original terms.

3.2.2 Method 2: Extractive PRF (CEQE)

This method, based on the **CEQE** paper, is an **extractive** pseudo-relevance feedback approach. We used a bi-encoder (`sentence-transformers/msmarco-bert-base-dot-v5`) to find terms in an initial set of retrieved documents that were semantically similar to the **full query context** (title + description + narrative). These extracted terms were then added to the original **title** with a tuned weight.

3.3 Dense Model Parameter Tuning

To find the optimal configuration for both dense models, we performed a sequential parameter sweep (coordinate ascent) to optimize for the best **F1@10** score.

3.3.1 Experiment 1: CEQE Tuning (Initial Sweep)

First, we performed a sweep to find the best `top_k_docs` to use for feedback.

Table 7: CEQE Performance vs. Top-K Feedback Docs (`n_terms=3`, `weight=20`)

<code>top_k_docs</code>	MRR@10	NDCG@10	P@10	F1@10
1	0.7784	0.4516	0.5367	0.1542
2	0.7856	0.4431	0.5233	0.1499
3	0.8259	0.4386	0.5067	0.1443
5	0.8051	0.4099	0.4833	0.1416
10	0.7611	0.3906	0.4433	0.1274

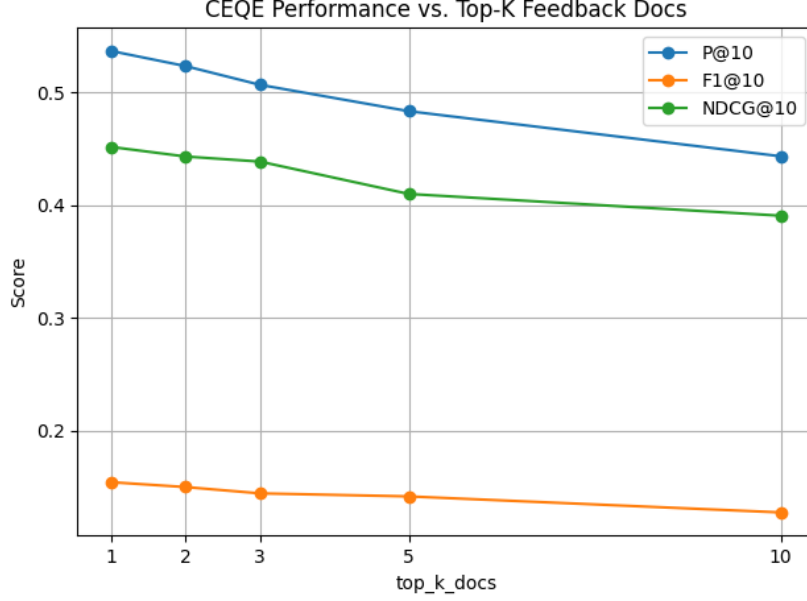


Figure 9: CEQE Performance vs. Top-K Feedback Docs

3.3.2 Experiment 2: CEQE Tuning (Robustness Check at k=3)

The initial sweep found $k_docs=1$ to be optimal, but we were concerned this result might be "brittle". We re-ran the tuning process, fixing $k_docs=3$ (a more robust choice).

Table 8: CEQE Performance vs. N Expansion Terms ($k_docs=3$, $weight=20$)

top_n_terms	MRR@10	NDCG@10	P@10	F1@10
1	0.7692	0.4268	0.5067	0.1425
3	0.8167	0.4759	0.5467	0.1591
5	0.7264	0.4290	0.5100	0.1463
10	0.7326	0.4256	0.5233	0.1526
15	0.7528	0.4051	0.4933	0.1403

This check proved successful, finding a new optimum at $n_terms=3$. We then tuned the query weight.

Table 9: CEQE Performance vs. Original Query Weight ($k_docs=3$, $n_terms=3$)

weight	MRR@10	NDCG@10	P@10	F1@10
1	0.7039	0.4038	0.4600	0.1321
5	0.7728	0.4539	0.5200	0.1484
10	0.7948	0.4636	0.5333	0.1528
15	0.7997	0.4703	0.5433	0.1581
20	0.8167	0.4759	0.5467	0.1591
25	0.7950	0.4681	0.5367	0.1571

This robustness check yielded our champion CEQE configuration: (**k_docs=3**, **n_terms=3**, **weight=20**).

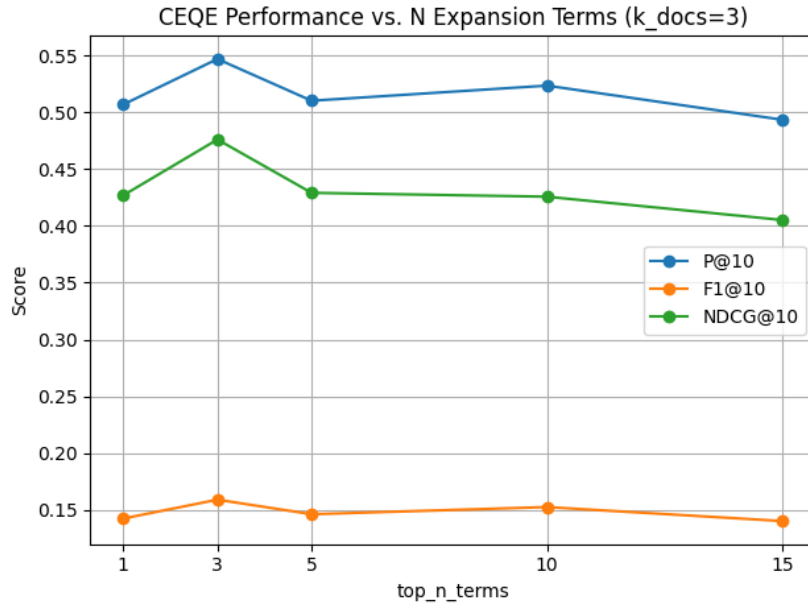


Figure 10: CEQE Performance vs. N Expansion Terms ($k_docs=3$)

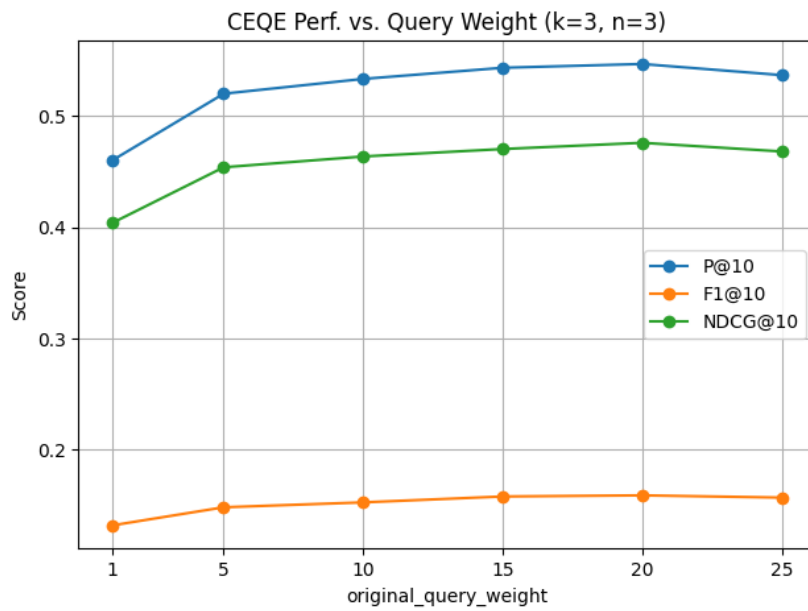


Figure 11: CEQE Performance vs. Query Weight ($k=3$, $n=3$)

3.3.3 Experiment 3: T5-Generative 2D Tuning

We performed a 2D grid search for the T5 model, tuning both the number of generated sequences and the original query weight. The F1@10 scores are reported below.

Table 10: T5-Generative F1@10 vs. Query Weight and Num. Sequences

Query Weight	1 seq	3 seqs	5 seqs
1	0.1079	0.0863	0.0812
5	0.1280	0.1086	0.1076
10	0.1342	0.1191	0.1165
15	0.1334	0.1258	0.1270
20	0.1342	0.1301	0.1310
25	0.1368	0.1333	0.1320

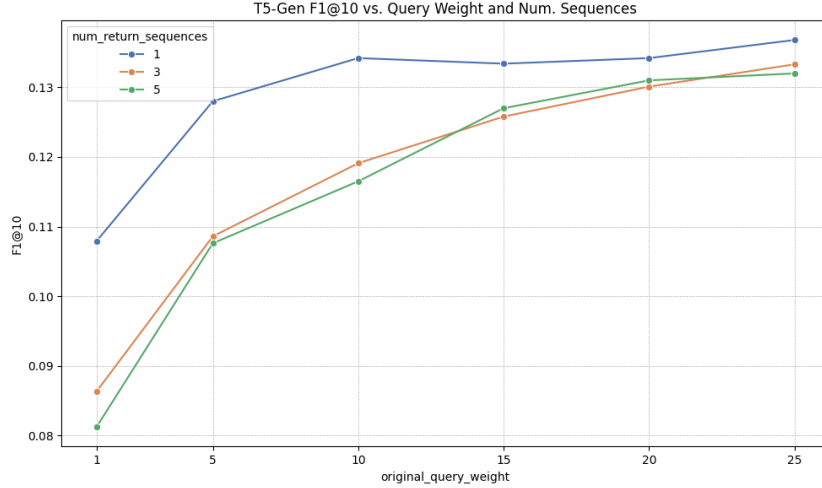


Figure 12: T5-Gen F1@10 vs. Query Weight and Num. Sequences

The T5 tuning revealed a clear trend: performance consistently improved with a higher `original_query_weight` and a **lower** `num_return_sequences`. The best T5 model, (**weight=25**, **num_seq=1**), was the one that added the **fewest** generated terms and gave them the **least** influence.

3.4 Final Experimental Results and Analysis

We compiled the final results from our four **champion** models: our best tuned baseline, our best tuned RM3, our best tuned CEQE, and our best tuned T5.

Table 11: Comprehensive performance comparsion on CORD-19

Run Name	P@1	P@10	R@10	F1@10	MRR@10	NDCG@10
BM25 (Default)	0.6333	0.4367	0.0720	0.1197	0.7187	0.3637
BM25 (Tuned)	0.6667	0.5167	0.0882	0.1458	0.7719	0.4304
RM3 (Default)	0.5000	0.4233	0.0796	0.1301	0.6381	0.3442
RM3 (Tuned)	0.6000	0.5000	0.0891	0.1462	0.7153	0.4252
QRF (CEQE-Best)	0.7333	0.5133	0.0856	0.1419	0.8045	0.4443
QRF (T5-Gen-Best)	0.7000	0.4800	0.0829	0.1368	0.7823	0.4012

3.5 Analysis and Discussion

Our final results were surprising and provided several key insights into retrieval on this scientific corpus.

- **Tuned Baselines are the Champions:** The most striking finding is that our two tuned baseline models, **BM25 (Tuned)** (F1@10: 0.1458) and **RM3 (Tuned)** (F1@10: 0.1462), were the overall winners. The **21.8% improvement** in F1@10 from tuning BM25 (0.1197 to 0.1458, from original full data) was the single largest performance gain of the entire experiment.
- **Dense Models Failed to Outperform:** Neither of our advanced, transformer-based methods could beat these simple baselines. **QRF (CEQE-Best)** (F1@10: 0.1419) and **QRF (T5-Gen-Best)** (F1@10: 0.1368) both underperformed.
- **Why did RM3 win?** Our default RM3 run failed catastrophically due to query drift. However, our **RM3 (Tuned)** model, with a highly conservative `original_query_weight=0.9`, solved this problem. It added just enough signal from the top-retrieved documents to **very slightly** improve recall (R@10: 0.0891 vs 0.0882) without harming precision, allowing it to narrowly beat the tuned BM25.
- **Why did the Dense Models Fail?**
 - **T5-Generative:** The 2D sweep for T5 showed that the model performed best when it added the **fewest possible terms** (`num_seq=1`) and gave them the **least influence** (`weight=25`). This implies the generated terms are low-quality. This is likely due to a **domain mismatch**: the `doc2query` model, trained on general web data, generates generic terms (e.g., "what is..") that are not helpful for the specific, scientific vocabulary of CORD-19.
 - **CEQE-Extractive:** CEQE’s failure (F1@10: 0.1419) is more subtle. While it achieved the best P@1 (0.7333) and MRR@10 (0.8045), indicating it’s very good at ranking the **single best document**, its performance at P@10 was lower than BM25. This suggests that even with our careful tuning, the model is still adding more noise than signal, or that the `msmarco-bert` bi-encoder is also suffering from a domain mismatch, struggling to identify the most salient scientific terms.
- **Note:** Due to extreme variability of times in repeated runs of the same models (possibly due to many teams working on the same VM), we weren’t able to perform a strong analysis of the runtimes of this task. However, we can say that the latency of our QRF techniques is around one order larger than BM25.

3.6 Task 3 Conclusion

Our investigation into query reformulation on the CORD-19 dataset yielded a clear and valuable conclusion: **fundamental baseline tuning is paramount, and simple models are hard to beat.**

The 22% improvement from tuning BM25 was the most significant gain in our experiment. Our attempts to improve this further with complex, transformer-based models (CEQE and T5) were unsuccessful. We found that both dense models likely suffered from a **domain mismatch** (being trained on general web text vs. a scientific corpus), which caused them to add more noise than signal.

Ultimately, the best-performing model was a simple, conservatively-tuned **RM3 (Tuned)**. This demonstrates that for this task, a well-tuned baseline with a minor, traditional pseudo-relevance feedback **nudge** was superior to more complex and computationally expensive dense methods.

References

- [1] Hofstätter, Sebastian and Lin, Sheng-Chieh and Yang, Jheng-Hong and Lin, Jimmy and Hanbury, Allan (2021). *Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling* In Proceedings of the 44th International ACM SIGIR Conference (SIGIR '21). [doi:10.1145/3404835.3462891](https://doi.org/10.1145/3404835.3462891)
- [2] Sanh, Victor and Debut, Lysandre and Chaumond, Julien and Wolf, Thomas (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108. [arXiv:1910.01108](https://arxiv.org/abs/1910.01108)
- [3] Thibault, T., et al. (2021). *SPLADE: Sparse Lexical and Expansion Model for First Stage Retrieval*. In Proceedings of the 44th International ACM SIGIR Conference (SIGIR '21). [doi:10.1145/3404835.3463098](https://doi.org/10.1145/3404835.3463098)
- [4] Thibault, T., et al. (2024). *SPLADE v2: Better, Faster and Lighter*. In Proceedings of the 47th International ACM SIGIR Conference (SIGIR '24). [doi:10.1145/3634912](https://doi.org/10.1145/3634912)